Additional Experiment for the paper

"Randomized Feasibility Methods for Constrained Optimization with Adaptive Step Sizes"

1 Constrained Logistic Regression with Fairness Constraints

We consider a constrained logistic regression problem with box constraints and group fairness constraints. Let $\{(a_j, y_j)\}_{j=1}^{\widehat{N}}$ denote the dataset, where $a_j \in \mathbb{R}^n$ are feature vectors and $y_j \in \{-1, +1\}$ are labels. The objective is to solve

$$\begin{aligned} \min_{x \in \mathbb{R}^n} \quad & f(x) := \frac{1}{N} \sum_{j=1}^{N} \log(1 + \exp(-y_j \langle a_j, x \rangle)) \\ \text{s.t.} \quad & x \in Y \cap X, \end{aligned}$$

where the set Y encodes simple box constraints, i.e., $Y := \{x \in \mathbb{R}^n \mid \ell \leq x \leq u\}$, and the set X encodes fairness constraints of the form

$$X := \bigcap_{g=1}^{m} \left\{ x \in \mathbb{R}^n \;\middle|\; \left| c_g^\top x \right| \leq \tau_g \right\}.$$

Each fairness constraint can equivalently be written as two convex inequalities:

$$c_g^\top x - \tau_g \leq 0, \qquad -c_g^\top x - \tau_g \leq 0.$$

For the synthetic data generation, we sample feature vectors $a_j \sim \mathcal{N}(0, I_n)$ and generate labels according to a logistic model

$$\mathbb{P}(y_j = 1 \mid a_j) = \sigma(a_j^\top \widehat{x}),$$

where $\sigma(t) = (1 + e^{-t})^{-1}$ and $\widehat{x} \in \mathbb{R}^n$ is a ground-truth vector sampled from a Gaussian distribution and projected onto the box $[\ell, u]$. The vectors $c_g \in \mathbb{R}^n$ are constructed from differences of group-level statistics. Specifically, suppose the dataset is partitioned into G groups indexed by $\mathcal{G}_1, \ldots, \mathcal{G}_G$. Define the group-wise empirical mean feature vectors

$$\mu_g := \frac{1}{|\mathcal{G}_g|} \sum_{j \in \mathcal{G}_g} a_j.$$

Then each fairness constraint is generated as $c_g := \mu_{g_1} - \mu_{g_2}$, for randomly selected pairs of groups (g_1, g_2). This leads to constraints enforcing similarity of model responses across groups:

$$\left| (\mu_{g_1} - \mu_{g_2})^\top x \right| \leq \tau_g.$$

We set the constraint thresholds as

$$\tau_g := \left| c_g^\top \widehat{x} \right| + \eta,$$

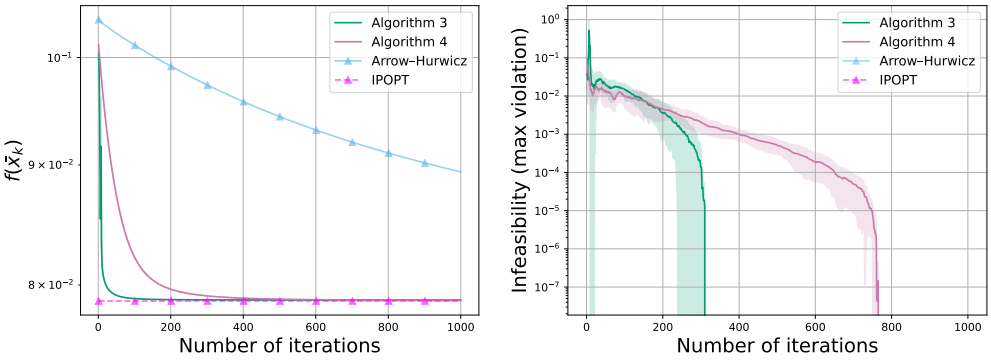
where $\eta > 0$ is a small slack parameter ensuring feasibility of $\widehat{x}$.

We ran our parameter-free Algorithms 3 (DoWS with Randomized Feasibility) and 4 (T-DoWS with Randomized Feasibility) and compared them with the Arrow-Hurwicz method. To obtain a baseline for the optimal function value, we also employed the state-of-the-art solver IPOPT. For data generation, we considered $\widehat{N} = 10{,}000$ samples with feature dimension $n = 500$, and partitioned the data into $G = 10$ groups.

Initially, we considered 3000 group constraints in the set X. However, IPOPT failed to converge within a reasonable time and frequently crashed. Since the IPOPT solution is used to estimate the baseline value

Table 1: Run time for a single experiment

Method	**Run time**
Arrow-Hurwicz	~ 8 secs
IPOPT	~ 77 secs
Algorithm 3 (Ours)	~ 17 secs
Algorithm 4 (Ours)	~ 17 secs



(a) Objective function value

(b) Maximum Infeasibility violation

Figure 1: Simulation plots for the constrained logistic regression with fairness constraints.

f^*, we reduced the number of group constraints to 1000 to ensure that IPOPT could successfully return a solution. We choose $N_k = 100 + \lceil k^{1/1.1} \rceil$ for our Algorithms 3 and 4. We report the function value and the max infeasibility violation across iterations in Figure 1. We also present the CPU runtime (on a MacBook Pro 2021 with an Apple M1 Pro chip and 16GB RAM) in Table 1.

From Figure 1, it is apparent that Algorithms 3 and 4 converge to the baseline optimal function value, matching the IPOPT solution. The convergence of the Arrow-Hurwicz method is relatively slow. Moreover, in terms of feasibility violations, IPOPT and Arrow-Hurwicz exhibit the best performance, while our algorithms also achieve zero feasibility violation after a sufficient number of iterations. It is worth noting that one can trade off between the growth schedule N_k for feasibility updates in our algorithms and feasibility violation, depending on the available computational time and memory constraints. A high N_k will result in a quick decrease in the infeasibility gap.